Создание таблиц

Теория

Разберём **ограничения** — правила, которые контролируют, какие данные можно вставить в таблицу.

Зачем нужны ограничения.
Без ограничений в таблицу можно вставить что угодно: пустое имя, отрицательную цену, дубликат email. Ограничения не дают этому случиться.

Основные ограничения.

Ограничение	Что делает	Пример
`NOT NULL`	Значение обязательно	`fio TEXT NOT NULL`
`UNIQUE`	Значения уникальны	`email TEXT UNIQUE`
`CHECK (условие)`	Значение удовлетворяет условию	`price REAL CHECK (price > 0)`
`DEFAULT значение`	Значение по умолчанию	`created TEXT DEFAULT (date('now'))`
`PRIMARY KEY`	Первичный ключ (NOT NULL + UNIQUE)	`id INTEGER PRIMARY KEY`
`FOREIGN KEY`	Ссылка на другую таблицу	`category_id INTEGER REFERENCES category(id)`

Пример с ограничениями:

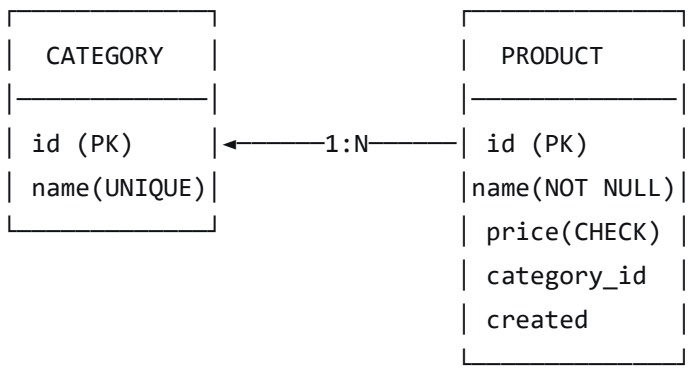
```
CREATE TABLE product (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  price REAL CHECK (price > 0),
  created TEXT DEFAULT (date('now')),
  sku TEXT UNIQUE
);
```

Разбор по столбцам:

- `id` — PRIMARY KEY AUTOINCREMENT — уникальный идентификатор.
- `name` — NOT NULL — имя обязательно.
- `price` — CHECK (price > 0) — цена должна быть положительной.
- `created` — DEFAULT (date('now')) — если не указать дату, подставится текущая.
- `sku` — UNIQUE — артикул не должен повторяться.

Про схемы.
В SQLite **нет схем** (в отличие от PostgreSQL). Все таблицы находятся в одной базе. Если нужно логически группировать таблицы — используют **префиксы**: `shop_product`, `shop_category`.

Диаграмма типичной схемы:



Типичные ошибки студентов.

1. **Путают NOT NULL и UNIQUE.** NOT NULL — «не пусто», UNIQUE — «не повторяется».
2. **Забывают CHECK.** Вставляют отрицательную цену и удивляются, что она сохранилась.
3. **Дублируют PRIMARY KEY и UNIQUE.** Если столбец PRIMARY KEY — он уже уникален, дополнительный UNIQUE не нужен.

Разобранный пример

Создадим таблицы `category` и `product`.

```
PRAGMA foreign_keys = ON;

CREATE TABLE category (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT UNIQUE NOT NULL
```

```
);

CREATE TABLE product (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  price REAL CHECK (price > 0),
  category_id INTEGER REFERENCES category(id),
  created TEXT DEFAULT (date('now'))
);

INSERT INTO category (name) VALUES ('Еда'), ('Быт');

INSERT INTO product (name, price, category_id) VALUES
('Хлеб', 40.00, 1),
('Мыло', 80.00, 2);
```

Что здесь важно:

- `category.name` — UNIQUE, значит, две категории с одинаковым названием создать нельзя.
- `product.price` — CHECK (price > 0), значит, отрицательную цену вставить нельзя.
- `product.category_id` — FOREIGN KEY на `category.id`.

Проверка:

```
INSERT INTO product (name, price) VALUES ('Плохой товар', -10);
```

Результат: ошибка `CHECK constraint failed`. Ограничение работает.

Задачи

1. Создайте таблицу `category` (id, name UNIQUE).
2. Создайте таблицу `product` со ссылкой на category.
3. Добавьте CHECK на цену > 0.
4. Вставьте по 2 записи в каждую таблицу.
5. Проверьте, что ограничения работают.
6. Создайте таблицу `supplier` (id, name, phone UNIQUE, email UNIQUE).
7. Создайте таблицу `shipment` (id, product_id FK, supplier_id FK, quantity CHECK > 0, date DEFAULT CURRENT_DATE).
8. Создайте таблицу с составным UNIQUE (product_id, supplier_id).

Задача «со звёздочкой»:

9. Создайте таблицу с CHECK, который проверяет дату (`CHECK (date >= '2020-01-01')`).

Задача на поиск ошибки:
Дан код:

```
CREATE TABLE product (
  id INTEGER PRIMARY KEY,
  name TEXT,
  price REAL CHECK (price > 0),
  category_id INTEGER REFERENCES category(id)
);
INSERT INTO product (name, price) VALUES ('Тест', 0);
```

Почему вставка не пройдёт?

Задача на предсказание результата:
Дана таблица category (id=1, name='Еда'). Что произойдёт при:

```
INSERT INTO product (name, price, category_id) VALUES ('Хлеб', 40, 1);
INSERT INTO product (name, price, category_id) VALUES ('Сыр', 200, 99);
```

если PRAGMA foreign_keys = ON?